Django replaces that with a single creation step that includes the added field. You can invoke this same optimization process on a single file that hasn't been squashed by using the `optimizemigration` management command. Personally, I've never used this; I do my best to keep my model changes as small as possible, leaving little need for optimization.

Finally, the `showmigrations` management command gives you detailed information on what migrations are in your project and which got run. You can, optionally, provide the name of one or more apps to filter the output. This is what `showmigrations` looks like after I rolled back the squashed 0006 migration:

```
(venv) RiffMates$ python manage.py showmigrations promoters
promoters
 [X] 0001_initial
 [X] 0002_promoter_famous_for
 [X] 0003_promoter_common_name_promoter_full_name
 [X] 0004_data_fullname
 [X] 0005_remove_promoter_first_name_remove_promoter_last_name
 [ ] 0006_promoter_birth_squashed_0007_promoter_death (2 squashed migrations)
```

Each `X` indicates migrations that have been run. For squashed migrations, the command even shows how many scripts got combined to create them. This command is helpful for looking at the current state of migration across your entire project at the same time.

10.5 Exercises

1. Modify the `Promoter` model by adding a "street address" field, migrating the change. Then, modify it again, adding fields for city, province/state, country, and postal/zip code. Create a migration script, migrate it, and then squash the migrations together.
2. Convert the address fields in exercise 1 into a field called *address*, a single multiline text field containing all the address information. Write migration scripts for adding the new field and converting any data from the old fields to the new ones. Finally, finish the process by removing the old fields and migrating that change as well.

Summary

- Migration scripts get generated by the `makemigrations` management command and are responsible for syncing the state of your `Model` files with the corresponding database tables.
- You use the `migrate` management command to invoke the migration scripts on the database.
- Migration scripts are Python programs that declare the `Migration` class, used to tell Django what changes need to be performed.